PASSWORD POLICY COMPLIANCE AUDITOR

PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Computer Applications (BCA)

Submitted By
Name: Sumit Gusain
Roll No.: R9PV45A72

Under the Guidance
Guide Name: Mr. Manish Singh

Department: Bachelor of Computer Application
University: Lovely Professional University
Summer Internship

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide for continuous guidance, motivation, and valuable suggestions throughout the development of this project. I am thankful to the Head of the Department and all faculty members for providing an excellent learning environment. I also thank my parents, friends, and classmates for their constant encouragement. I acknowledge the contribution of the open-source community, the Python ecosystem, the Flask framework, and the organizations NIST and OWASP whose published standards helped shape this project. Their resources enabled me to understand password security, secure authentication, and compliance auditing in a practical manner.

I express my sincere gratitude to my project guide for continuous guidance, motivation, and valuable suggestions throughout the development of this project. I am thankful to the Head of the Department and all faculty members for providing an excellent learning environment. I also thank my parents, friends, and classmates for their constant encouragement. I acknowledge the contribution of the open-source community, the Python ecosystem, the Flask framework, and the organizations NIST and OWASP whose published standards helped shape this project. Their resources enabled me to understand password security, secure authentication, and compliance auditing in a practical manner.

I express my sincere gratitude to my project guide for continuous guidance, motivation, and valuable suggestions throughout the development of this project. I am thankful to the Head of the Department and all faculty members for providing an excellent learning environment. I also thank my parents, friends, and classmates for their constant encouragement. I acknowledge the contribution of the open-source community, the Python ecosystem, the Flask framework, and the organizations NIST and OWASP whose published standards helped shape this project. Their resources enabled me to understand password security, secure authentication, and compliance auditing in a practical manner.

ABSTRACT

Passwords remain the primary authentication mechanism for most digital systems, making password security a critical aspect of cybersecurity. Weak passwords are a common cause of unauthorized access and data breaches. The Password Policy Compliance Auditor is designed to evaluate passwords against organizational policies based on NIST SP 800-63B and OWASP Password Guidelines. The application validates password length, character diversity, common password usage, repeated patterns, and overall compliance. It is developed using Python and Flask, uses SQLite to store audit records, bcrypt for secure password hashing, and ReportLab for PDF report generation. The application provides users with compliance status, security recommendations, and downloadable reports. The proposed system helps organizations improve password hygiene, promote secure password practices, and reduce risks associated with weak credentials.

Keywords: Password Security, Password Policy, NIST SP 800-63B, OWASP, Flask, SQLite, bcrypt, Cybersecurity.

TABLE OF CONTENTS

1. Acknowledgement

2. Abstract

3. Table of Contents

4. Introduction

7. Objectives

8. Methodology

9. Data Analysis & Interpretation

10.Discussion and Results

11. Conclusion

12. Recommendations & Suggestions

13. Bibliography/References

14. Appendices/Annexures

Introduction

Passwords are the most common way of protecting user accounts and sensitive information. However, many users create weak passwords that are easy to guess or crack. Weak passwords can lead to unauthorized access, data theft, and cyberattacks.

The **Password Policy Compliance Auditor** is a Python-based application that checks whether a password follows secure password policies. The project is based on the security recommendations provided by **NIST SP 800-63B** and the **OWASP Password Guidelines**.

The system evaluates passwords using different security rules, such as minimum length, use of uppercase and lowercase letters, numbers, special characters, and checks for common or weak passwords. Based on these checks, it determines whether the password is **Compliant** or **Non-Compliant**.

The application is developed using **Python** with the **Flask** framework. **SQLite** is used to store user and audit information, while **bcrypt** is used to securely hash passwords before storing them. The system also generates a PDF compliance report containing the password evaluation results and suggestions for improvement.

The main objective of this project is to help users create stronger passwords, improve password security, and encourage organizations to follow standard password policies. By automatically checking password compliance, the system reduces the risk of security breaches caused by weak passwords and promotes better cybersecurity practices.

Objective

The main objective of the **Password Policy Compliance Auditor** is to develop a secure and user-friendly application that evaluates passwords based on recognized security standards such as **NIST SP 800-63B** and the **OWASP Password Guidelines**. The system is designed to identify weak and non-compliant passwords by checking important security parameters, including password length, character diversity, common password usage, and predictable patterns. It aims to help users create stronger passwords by providing immediate feedback and practical recommendations for improvement. The application securely stores passwords using the **bcrypt** hashing algorithm, maintains audit records in an **SQLite** database, and generates professional PDF compliance reports for documentation and analysis. Through automated password auditing, the project seeks to reduce the risk of unauthorized access, improve password security awareness, promote secure authentication practices, and assist organizations in enforcing effective password policies to enhance overall cybersecurity.

Problem Statement

In today's digital world, passwords are the primary method used to protect user accounts and sensitive information. However, many users continue to create weak, simple, or commonly used passwords that are easy for attackers to guess or crack. Weak passwords significantly increase the risk of cyberattacks such as brute-force attacks, dictionary attacks, credential stuffing, and unauthorized access, leading to data breaches and financial losses.

Many organizations define password policies to improve security, but ensuring that every password complies with these policies is often difficult. Manual verification is time-consuming, inconsistent, and prone to human error. In addition, many existing password strength checkers only provide a basic strength score and do not verify compliance with recognized security standards such as **NIST SP 800-63B** and the **OWASP Password Guidelines**. They also lack features such as audit history, secure password handling, and professional compliance reporting.

To address these challenges, there is a need for an automated system that can evaluate passwords against established security standards, identify weak credentials, and provide clear recommendations for improvement. The **Password Policy Compliance Auditor** has been developed to solve this problem by automatically validating passwords based on predefined security rules, securely handling password data using the **bcrypt** hashing algorithm, maintaining audit records in an **SQLite** database, and generating detailed PDF compliance reports. The system helps users create stronger passwords, supports organizations in enforcing password policies, and contributes to improving overall cybersecurity by promoting secure password practices.

Methodology

The development of the **Password Policy Compliance Auditor** follows a systematic software development methodology to ensure that the application is secure, reliable, and easy to use. The project is implemented using **Python** as the programming language, **Flask** as the web framework, **SQLite** as the database, and **bcrypt** for secure password hashing. The password validation rules are based on the recommendations of **NIST SP 800-63B** and the **OWASP Password Guidelines**.

The development process begins with the **requirement analysis** phase, where the project objectives, user requirements, and password security standards are studied. During this phase, the functional requirements such as user registration, login, password auditing, report generation, and audit history are identified. Non-functional requirements, including security, performance, reliability, and usability, are also considered.

The next phase is **system design**, where the overall architecture of the application is planned. The system is divided into different modules such as user authentication, password compliance checking, database management, and PDF report generation. Database tables are designed to store user information, audit history, and generated reports efficiently.

After the design phase, the **implementation** phase begins. The application is developed using the Flask framework, while SQLite is used to manage the database. Passwords are securely hashed using the bcrypt algorithm before being stored, ensuring that plain-text passwords are never saved. The password compliance module checks each password against predefined rules, including minimum length, maximum length, uppercase and lowercase letters, numbers, special characters, common passwords, repeated characters, and sequential patterns. Based on these checks, the system calculates a compliance score and classifies the password as **Compliant** or **Non-Compliant**.

The **testing** phase is conducted to verify that all modules work correctly. Different types of passwords are tested to ensure that the application

accurately identifies strong and weak passwords. Functional testing is performed for user registration, login, password validation, report generation, and database operations. Any errors identified during testing are corrected to improve the reliability of the system.

Finally, the **deployment and documentation** phase is carried out. The completed application is prepared for execution on a local computer using Python and Flask. A detailed project report is prepared, explaining the system design, implementation, testing process, results, and conclusions. This methodology ensures that the Password Policy Compliance Auditor is developed in a structured manner while meeting modern cybersecurity standards and promoting secure password practices.

Data Analysis & Interpretation

The **Data Analysis and Interpretation** phase evaluates the effectiveness of the **Password Policy Compliance Auditor** by analyzing how the application validates passwords against the security requirements defined by **NIST SP 800-63B** and the **OWASP Password Guidelines**. The purpose of this analysis is to determine whether the system accurately identifies compliant and non-compliant passwords and provides meaningful recommendations to improve password security.

Several passwords with different combinations of characters, lengths, and complexity levels were tested. The system examined each password based on criteria such as minimum password length, presence of uppercase and lowercase letters, numeric digits, special characters, common password detection, and repeated or sequential characters. After completing the evaluation, the application calculated a compliance score and classified each password as either **Compliant** or **Non-Compliant**.

The analysis showed that passwords containing a combination of uppercase letters, lowercase letters, numbers, special characters, and sufficient length consistently achieved higher compliance scores and were classified as **Compliant**. On the other hand, passwords that were short, commonly used, or lacked character diversity received lower scores and were marked as **Non-Compliant**. For every non-compliant password, the application generated clear recommendations, such as increasing the password length, avoiding common words, adding numbers and special characters, and creating more unique passwords.

The audit records stored in the **SQLite** database confirmed that every password evaluation was successfully recorded along with the compliance status, security score, identified weaknesses, and audit date. Additionally, the system successfully generated PDF compliance reports containing detailed evaluation results, which can be used for documentation and future reference.

Discussion and Results

The **Password Policy Compliance Auditor** was successfully designed and implemented using **Python** as the programming language. The project uses the **Flask** framework for web application development, **SQLite** for database management, and **bcrypt** for secure password hashing. The application evaluates passwords according to the security recommendations provided by **NIST SP 800-63B** and the **OWASP Password Guidelines**. The main objective of the system is to identify weak passwords, encourage users to create stronger passwords, and help organizations enforce secure password policies.

The developed system was tested using various passwords with different lengths and combinations of uppercase letters, lowercase letters, numbers, and special characters. The testing results showed that the application successfully validated passwords against predefined security rules. Passwords that met all the required criteria were classified as **Compliant**, while passwords that failed one or more validation rules were marked as **Non-Compliant**. The system also provided clear suggestions to help users improve weak passwords by increasing their length, using different character types, and avoiding common or predictable passwords.

The password compliance engine performed accurately by checking password length, character complexity, repeated characters, sequential patterns, and common passwords. Passwords that satisfied all validation rules received higher compliance scores, whereas weak passwords received lower scores. This demonstrates that the system effectively distinguishes between secure and insecure passwords.

The application also successfully stored audit records in the **SQLite** database. Instead of saving passwords in plain text, the system securely hashed every password using the **bcrypt** algorithm before storing it. This approach enhances data security and follows modern password storage practices. In addition, the system generated PDF compliance reports containing the audit date, compliance status, security score, identified

weaknesses, and recommendations for improvement. These reports can be used for documentation and future reference.

Overall, the project achieved all its intended objectives. The application successfully validates password policies, identifies weak credentials, securely stores password information, maintains audit records, and generates professional compliance reports. The results indicate that the **Password Policy Compliance Auditor** is an effective and reliable solution for improving password security, promoting secure password practices, and helping organizations comply with internationally recognized cybersecurity standards. The project demonstrates the practical application of password security concepts and provides a useful tool for enhancing overall cybersecurity awareness and reducing the risks associated with weak passwords.

Conclusion

The **Password Policy Compliance Auditor** was successfully developed to evaluate passwords according to the security recommendations of **NIST SP 800-63B** and the **OWASP Password Guidelines**. The project achieved its primary objective of identifying weak and non-compliant passwords while encouraging users to create stronger and more secure passwords.

The application was implemented using **Python** with the **Flask** framework, **SQLite** for database management, and **bcrypt** for secure password hashing. It validates passwords based on important security parameters such as password length, uppercase and lowercase letters, numeric digits, special characters, and common password detection. Based on these checks, the system classifies passwords as **Compliant** or **Non-Compliant** and provides useful recommendations for improvement. In addition, the application securely stores audit records and generates professional PDF compliance reports for documentation and future reference.

The testing results confirmed that the system accurately evaluates password strength and successfully detects weak credentials. The use of bcrypt ensures that passwords are stored securely, while the PDF reporting feature makes the application suitable for organizations that need to maintain password compliance records. The project demonstrates the practical implementation of cybersecurity concepts and highlights the importance of following standard password policies to reduce the risk of unauthorized access and cyberattacks.

Overall, the **Password Policy Compliance Auditor** provides a simple, reliable, and effective solution for password security assessment. It helps users improve password quality, supports organizations in enforcing password policies, and promotes better cybersecurity practices. The project successfully meets its objectives and serves as a useful tool for enhancing password security and compliance with internationally recognized standards.

Recommendations & Suggestions

The following recommendations are suggested to improve the effectiveness and future usability of the **Password Policy Compliance Auditor**:

1. Organizations should implement strong password policies based on **NIST SP 800-63B** and **OWASP Password Guidelines** to improve account security and reduce the risk of cyberattacks.
2. Users should create long and unique passwords that include a combination of uppercase letters, lowercase letters, numbers, and special characters while avoiding common or easily predictable passwords.
3. Passwords should always be stored using secure hashing algorithms such as **bcrypt** instead of plain text to protect user credentials in case of a database breach.
4. The system should be updated regularly to include the latest password security standards and newly discovered weak or compromised passwords.
5. Future versions of the application can integrate **Multi-Factor Authentication (MFA)** to provide an additional layer of security beyond passwords.
6. The application can be enhanced by integrating online breached-password databases, such as **Have I Been Pwned**, to detect passwords that have already been exposed in data breaches.
7. An administrator dashboard can be added to monitor user compliance, generate analytical reports, and manage password policies across an organization.
8. Cloud database support and role-based access control can be implemented to make the application suitable for large organizations and enterprise environments.
9. Regular security awareness training should be provided to users so they understand the importance of creating strong passwords and following secure authentication practices.
10. Continuous testing, maintenance, and software updates should be performed to ensure the application remains secure, reliable, and compatible with evolving cybersecurity requirements.

Bibliography / References

The following books, research papers, official standards, and online resources were referred to during the development of the **Password Policy Compliance Auditor** project.

Books

1. Cryptography and Network Security: Principles and Practice, Pearson Education, 2020.
2. Computer Security: Principles and Practice, Pearson Education, 2018.
3. Python Crash Course, No Starch Press, 2023.
4. Flask Web Development, O'Reilly Media, 2018.

Official Standards

1. NIST Special Publication 800-63B, National Institute of Standards and Technology (NIST).
2. OWASP Password Guidelines, OWASP Foundation.
3. OWASP Password Storage Cheat Sheet, OWASP Foundation.
4. OWASP Authentication Cheat Sheet, OWASP Foundation.

Online References

1. NIST Digital Identity Guidelines (SP 800-63B)
2. OWASP Official Website
3. Python Documentation
4. Flask Documentation
5. SQLite Documentation
6. SQLAlchemy Documentation

7. ReportLab Documentation
8. bcrypt Documentation
9. GitHub Official Website
10. Visual Studio Code Documentation

Research Papers

1. A Future-Adaptable Password Scheme, USENIX Annual Technical Conference, 1999.
2. The Memorability and Security of Passwords.

Appendices/Annexures

Appendix A :Installation Guide

1. Install Python 3.
2. Create a virtual environment.
3. Install dependencies using requirements.txt.
4. Run the Flask application.
5. Open the application in a web browser.
6. Register a user and perform password audits.

Appendix B: User Manual

1. Register a new account.
2. Login with valid credentials.
3. Navigate to the Password Audit page.
4. Enter a password for evaluation.
5. Review the compliance score and recommendations.
6. Generate and download the PDF report.
7. View audit history.

Appendix C: Source Code Modules

1. app.py - Main Flask application
2. models.py - Database models
3. routes.py - URL routes
4. auditor.py - Password validation engine
5. report.py - PDF generation
6. templates/ - HTML pages
7. static/ - CSS and JavaScript resources

Appendix D: Project Outcomes

The Password Policy Compliance Auditor validates passwords against organizational policies, highlights weak credentials, promotes secure password practices, stores audit history securely, and generates professional PDF compliance reports. The project demonstrates practical implementation

of NIST SP 800-63B and OWASP Password Guidelines using Python, Flask, SQLite, and bcrypt.